

Fourth Assignment: Exact Algorithms for Combinatorial Optimization

Solution

1 Exact Algorithms in Optimization

Conceptual Question 1

What is an exact algorithm?

An *exact algorithm* for an optimization (or decision) problem is an algorithm that always returns a *provably optimal* solution (or correct yes/no answer) whenever a solution exists, and correctly reports infeasibility otherwise, typically within finite time for each fixed input instance.

Conceptual Question 2

Difference between exact vs. heuristic / approximation algorithms

- Exact algorithms guarantee to find an optimal solution, or prove optimality or infeasibility, for every instance; they often have exponential or pseudo-polynomial worst-case complexity on NP-hard problems.
- Heuristic algorithms use problem-specific rules or local search to quickly obtain *good* solutions but offer no general worst-case guarantee on closeness to optimality.
- Approximation algorithms return solutions with a provable performance ratio relative to the optimum, but need not attain the exact optimum.

Conceptual Question 3

Role of bounds and relaxations in exact algorithms

- *Relaxations* (for example, LP relaxations of integer programs) replace hard constraints by easier ones, giving a problem whose optimal value is a bound on the original problem.
- *Bounds* (upper bounds for maximization, lower bounds for minimization) are used in exact search methods to prune parts of the search tree that cannot contain better solutions than the best incumbent.
- Strong relaxations and tight bounds are crucial for the efficiency of branch-and-bound, branch-and-cut, Benders decomposition, Dantzig–Wolfe decomposition, and column generation.

2 Complete Enumeration (Brute Force)

Conceptual Question 1

What is complete enumeration?

Complete enumeration (brute force) is the method that systematically generates and evaluates *all* feasible solutions of a problem, selecting the best one for optimization, or checking all candidates until a required property is found.

Conceptual Question 2

Computational complexity of brute force for combinatorial problems

For NP-hard combinatorial problems, the number of feasible solutions typically grows exponentially with the input size, so brute-force search requires time exponential in the problem size, for example $O(2^n)$ for a 0/1 problem with n binary variables.

Conceptual Question 3

Why is brute force rarely practical for large-scale problems?

Because the search space grows exponentially, the number of candidate solutions becomes astronomically large even for moderate problem sizes, leading to prohibitive runtimes and memory usage on realistic instances, so brute force is limited to very small or highly constrained problems.

Conceptual Question 4

When can brute force still be useful?

- For very small instances where the solution space is tiny.
- As a benchmark to evaluate heuristic or approximation algorithms on small test cases by providing the true optimal value.
- When the structure severely restricts feasible solutions so that the actual number of candidates is limited.

Exercise: 0/1 Knapsack

Problem:

$$\max 8x_1 + 6x_2 + 5x_3$$

subject to

$$4x_1 + 3x_2 + 2x_3 \leq 5, \quad x_i \in \{0, 1\}.$$

1. All feasible solutions. There are $2^3 = 8$ binary vectors (x_1, x_2, x_3) . For each, we check the weight constraint.

(x_1, x_2, x_3)	$4x_1 + 3x_2 + 2x_3$	Feasible?
(0, 0, 0)	0	yes
(1, 0, 0)	4	yes
(0, 1, 0)	3	yes
(0, 0, 1)	2	yes
(1, 1, 0)	7	no
(1, 0, 1)	6	no
(0, 1, 1)	5	yes
(1, 1, 1)	9	no

Thus the feasible solutions are

$$(0, 0, 0), (1, 0, 0), (0, 1, 0), (0, 0, 1), (0, 1, 1).$$

2. Objective value of each feasible solution.

(x_1, x_2, x_3)	$8x_1 + 6x_2 + 5x_3$
(0, 0, 0)	0
(1, 0, 0)	8
(0, 1, 0)	6
(0, 0, 1)	5
(0, 1, 1)	11

3. Optimal solution. The maximum profit is 11, attained at $(x_1, x_2, x_3) = (0, 1, 1)$, so the optimal solution is

$$x_1^* = 0, \quad x_2^* = 1, \quad x_3^* = 1$$

with objective value $z^* = 11$.

4. Number of solutions to check in general.

- (i) 0/1 knapsack with n items: there are 2^n possible subsets, so brute force must check up to 2^n solutions.
- (ii) Traveling Salesperson Problem (TSP) with n cities: the number of Hamiltonian tours is $(n-1)!/2$ for an undirected symmetric TSP, so an exhaustive search checks $\Theta(n!)$ tours.
- (iii) Given IP:

$$\begin{aligned}
\min \quad & 4x_1 + 6x_2 + 2x_3 + x_4 + 3x_5 \\
\text{s.t.} \quad & x_1 + 2x_2 + x_3 \geq 4, \\
& 3x_1 + x_4 + x_5 \leq 25, \\
& x_2 + x_3 + 2x_5 \geq 8, \\
& x_3 + x_4 \leq 20, \\
& x_1, x_2 \in \{0, 1\}, \\
& x_3 \in \{1, \dots, 10\}, \\
& x_4, x_5 \in \{j \in \mathbb{Z} \mid j \text{ odd}, 0 < j < 20\}.
\end{aligned}$$

- x_1 : 2 values,
- x_2 : 2 values,

- x_3 : 10 values,
- x_4 : odd integers in $\{1, 3, \dots, 19\}$, so 10 values,
- x_5 : same as x_4 , so 10 values.

Hence a naive enumeration over the unrestricted domains would consider

$$2 \cdot 2 \cdot 10 \cdot 10 \cdot 10 = 4000$$

candidate assignments.

3 Dynamic Programming

Conceptual Question 1

Main idea behind dynamic programming

Dynamic programming solves optimization problems by decomposing them into overlapping subproblems, solving each smaller subproblem once, and storing its solution in a table so it can be reused instead of recomputed.

Conceptual Question 2

Overlapping subproblems

Overlapping subproblems occur when the same subproblem, or the same state, is encountered multiple times in the natural recursive formulation, so that memoization or tabulation leads to significant savings compared with naive recursion.

Conceptual Question 3

Principle of optimality

Bellman's principle of optimality states that an optimal policy has the property that, regardless of the initial state and initial decision, the remaining decisions must constitute an optimal policy with respect to the state resulting from the first decision.

Conceptual Question 4

Types of optimization problems suitable for DP

Problems are well suited for dynamic programming when they exhibit

- optimal substructure, and
- overlapping subproblems.

Typical examples include shortest paths, knapsack, sequence alignment, and many scheduling and inventory problems.

Conceptual Question 5

Components of a dynamic program

A dynamic program typically consists of:

- *States*: variables describing the subproblem.
- *Decision variables*: choices made at each state.
- *Recurrence (Bellman) equation*: expresses the value of a state in terms of smaller states.
- *Boundary / initialization*: base cases.
- *Evaluation order*: an ordering that ensures predecessor states are already computed.
- *Reconstruction rule*: how to retrieve an optimal solution from the table.

Exercise: Knapsack DP

Profits $p = (6, 4, 5)$, weights $w = (3, 2, 4)$, capacity $W = 5$.

1. Recurrence relation. Let $n = 3$ and define

$V(i, c)$ = maximum profit using items $1, \dots, i$ with capacity c .

Then for $i \geq 1$ and $0 \leq c \leq W$:

$$V(i, c) = \begin{cases} V(i-1, c), & w_i > c, \\ \max\{V(i-1, c), V(i-1, c - w_i) + p_i\}, & w_i \leq c, \end{cases}$$

with base conditions $V(0, c) = 0$ for all c and $V(i, 0) = 0$ for all i .

2. Components of the DP.

- States: (i, c) where $i \in \{0, \dots, 3\}$ and $c \in \{0, \dots, 5\}$.
- Decision at each state: include item i or not, if $w_i \leq c$.
- Recurrence: as given above.
- Boundary conditions: $V(0, c) = 0$ and $V(i, 0) = 0$.
- Evaluation order: increasing i from 1 to 3, and for each i , increasing c from 0 to 5.
- Reconstruction: starting from $(i, c) = (3, 5)$, compare $V(i, c)$ with $V(i-1, c)$; if equal then $x_i = 0$, otherwise $x_i = 1$ and set $c \leftarrow c - w_i$.

3. DP table. We build the table $V(i, c)$:

$V(i, c)$	$c = 0$	1	2	3	4	5
$i = 0$	0	0	0	0	0	0
$i = 1$	0	0	0	6	6	6
$i = 2$	0	0	4	6	6	10
$i = 3$	0	0	4	6	6	10

Compute rows:

- $i = 1, w_1 = 3, p_1 = 6$: for $c < 3$ we cannot take item 1; for $c \geq 3, V(1, c) = 6$.
- $i = 2, w_2 = 2, p_2 = 4$: for $c < 2$ cannot take; at $c = 2, V(2, 2) = 4$; at $c = 3, V(2, 3) = 6$; at $c = 5, V(2, 5) = 10$.
- $i = 3, w_3 = 4, p_3 = 5$: for $c < 4$ cannot take; at $c = 4, V(3, 4) = 6$; at $c = 5, V(3, 5) = 10$.

4. Optimal solution. The optimal value is $V(3, 5) = 10$.

Recovering the items:

- $(i, c) = (3, 5)$: $V(3, 5) = 10$ and $V(2, 5) = 10$, so item 3 is not chosen.
- $(i, c) = (2, 5)$: $V(2, 5) = 10$ and $V(1, 5) = 6 \neq 10$, so item 2 is chosen; new capacity $c = 3$.
- $(i, c) = (1, 3)$: $V(1, 3) = 6$ and $V(0, 3) = 0 \neq 6$, so item 1 is chosen.

Thus an optimal solution is

$$x_1^* = 1, \quad x_2^* = 1, \quad x_3^* = 0, \quad z^* = 10.$$

5. Time complexity of knapsack DP. The usual table has dimensions $(n+1) \times (W+1)$ and each entry is computed in $O(1)$ time, so the complexity is $O(nW)$, which is pseudo-polynomial in the input size.

6. What is forward dynamic programming? Forward dynamic programming processes the states in the natural time or stage order from the beginning to the end, propagating information forward through the state space, rather than backward from a terminal condition.

7. Components of forward DP. For forward DP on knapsack we can define:

- States: capacity level c after considering a certain stage, or more generally (i, c) .
- Transition: from (i, c) to $(i + 1, c)$ by skipping item $i + 1$, or to $(i + 1, c + w_{i+1})$ by adding item $i + 1$, provided feasibility is maintained.
- Initial state: $(0, 0)$.
- Value function: best profit reachable at each state.
- Termination: after processing all items, inspect the states with capacity $\leq W$ to extract the best profit and decision sequence.

8. Comparison of the two DPs.

- Backward DP, such as the classical $V(i, c)$ recurrence, references previous stages.
- Forward DP focuses on reachable states as we move from earlier to later stages.
- Both have similar worst-case complexity $O(nW)$ and rely on the same dynamic programming principle.

4 Branch-and-Bound

Conceptual Question 1

Main idea of Branch-and-Bound

Branch-and-Bound (B&B) is an exact tree search algorithm that recursively partitions the feasible region into subproblems (branching) and uses bounds from relaxations to discard subproblems that cannot contain a better solution than the best incumbent.

Typical steps:

- Solve a relaxation of the current subproblem to obtain a bound.
- If the relaxation is infeasible or its bound is worse than the incumbent, prune the node.
- If the relaxation yields a feasible integer solution better than the incumbent, update the incumbent.
- Otherwise, select a branching variable and create child subproblems with additional constraints.

Conceptual Question 2

Three fundamental components

- *Branching*: partition the feasible region into disjoint subproblems.
- *Bounding*: compute lower or upper bounds on the best objective value attainable within a node.
- *Node selection and pruning*: choose which node to explore next and discard nodes based on bounds or infeasibility.

Conceptual Question 3

What does it mean to prune a node?

To prune a node means to stop exploring that subproblem and discard its descendants because the node is either infeasible, already worse than the incumbent, or yields an integer solution that has been fully processed.

Conceptual Question 4

Why can B&B dramatically reduce the search space?

Strong bounds allow many subproblems to be pruned without exhaustive exploration, which can reduce the effective search tree from exponential size to a much smaller number of nodes in practice.

Exercise: B&B for a small IP

$$\begin{aligned} \max \quad & 4x_1 + 3x_2 \\ \text{s.t.} \quad & 2x_1 + x_2 \leq 7, \\ & x_1 + 2x_2 \leq 7, \\ & x_1, x_2 \in \mathbb{Z}_{\geq 0}. \end{aligned}$$

1. LP relaxation. Relax integrality:

$$x_1, x_2 \geq 0, \quad \text{continuous.}$$

We solve

$$\max 4x_1 + 3x_2 \quad \text{s.t.} \quad 2x_1 + x_2 \leq 7, \quad x_1 + 2x_2 \leq 7, \quad x_1, x_2 \geq 0.$$

The intersection of

$$2x_1 + x_2 = 7, \quad x_1 + 2x_2 = 7$$

gives

$$x_1 = x_2 = \frac{7}{3}.$$

Corner points:

$$(0, 0), \quad (3.5, 0), \quad (0, 3.5), \quad \left(\frac{7}{3}, \frac{7}{3}\right).$$

Evaluate the objective:

(x_1, x_2)	$4x_1 + 3x_2$
$(0, 0)$	0
$(3.5, 0)$	14
$(0, 3.5)$	10.5
$\left(\frac{7}{3}, \frac{7}{3}\right)$	$\frac{49}{3} \approx 16.33$

Thus the LP optimum is at

$$(x_1, x_2) = \left(\frac{7}{3}, \frac{7}{3}\right)$$

with value

$$\frac{49}{3}.$$

2. Branch-and-Bound search tree. The fractional solution has both variables fractional. We branch on x_1 .

- Root node (Node 0): LP solution $\left(\frac{7}{3}, \frac{7}{3}\right)$, bound $\frac{49}{3}$.
- Node 1: add constraint $x_1 \leq 2$.
- Node 2: add constraint $x_1 \geq 3$.

Node 1: $x_1 \leq 2$. Solve

$$\max 4x_1 + 3x_2$$

subject to

$$2x_1 + x_2 \leq 7, \quad x_1 + 2x_2 \leq 7, \quad 0 \leq x_1 \leq 2, \quad x_2 \geq 0.$$

Consider:

- $(2, 0)$: objective 8.
- Intersection of $x_1 = 2$ and $x_1 + 2x_2 = 7$: gives $x_2 = 2.5$, objective 15.5.
- Intersection of $x_1 = 2$ and $2x_1 + x_2 = 7$: gives $x_2 = 3$, but this violates $x_1 + 2x_2 \leq 7$.

So the LP optimum at Node 1 is $(2, 2.5)$ with value 15.5. It is fractional, so branch on x_2 :

- Node 1a: $x_1 \leq 2$ and $x_2 \leq 2$.
- Node 1b: $x_1 \leq 2$ and $x_2 \geq 3$.

Node 1a: $x_1 \leq 2, x_2 \leq 2$. Among integer pairs with $x_1 \in \{0, 1, 2\}$ and $x_2 \in \{0, 1, 2\}$, the point

$$(2, 2)$$

is feasible and gives

$$z = 4 \cdot 2 + 3 \cdot 2 = 14.$$

So Node 1a yields an integer feasible solution with objective value 14. Update the incumbent to

$$z^* = 14.$$

Node 1b: $x_1 \leq 2, x_2 \geq 3$. Possible integer points include

$$(0, 3) \quad \text{and} \quad (1, 3).$$

Their objective values are

$$(0, 3) : z = 9, \quad (1, 3) : z = 13.$$

So this node cannot improve the incumbent 14 and is pruned.

Node 2: $x_1 \geq 3$. Now solve with $x_1 \geq 3$:

$$2x_1 + x_2 \leq 7, \quad x_1 + 2x_2 \leq 7, \quad x_1 \geq 3, \quad x_2 \geq 0.$$

Feasible integer points include

$$(3, 0), \quad (3, 1).$$

Objective values:

$$(3, 0) : z = 12, \quad (3, 1) : z = 15.$$

The point $(3, 1)$ is feasible and integral, so it improves the incumbent:

$$z^* = 15.$$

3. Bounds at nodes.

- Node 0: bound $\frac{49}{3} \approx 16.33$.
- Node 1: bound 15.5.
- Node 1a: bound 14.
- Node 1b: best value 13.
- Node 2: bound 15.

4. Pruned nodes.

- Node 1a is pruned because it yields an integer solution for its subtree.
- Node 1b is pruned because its best possible value does not exceed the incumbent.

5. Optimal integer solution.

The best integer solution found is

$$x_1^* = 3, \quad x_2^* = 1, \quad z^* = 15.$$

5 Constraint Programming

Conceptual Question 1

What is Constraint Programming?

Constraint Programming (CP) is a paradigm for solving combinatorial problems by stating them as constraint satisfaction or optimization problems over variables with specified domains, and using systematic search together with inference, or constraint propagation, to prune infeasible values.

Conceptual Question 2

Difference between CP and Mixed Integer Programming (MIP)

- MIP uses linear inequalities and an objective function, relying heavily on LP relaxations, polyhedral theory, and branch-and-bound.
- CP allows rich, high-level global constraints and nonlinear logical constraints over discrete domains, with propagation algorithms tailored to each constraint.

Conceptual Question 3

What is a CSP?

A constraint satisfaction problem (CSP) consists of a set of variables, each with a finite domain of possible values, and a set of constraints specifying allowed combinations of values. The goal is to assign each variable a value from its domain so that all constraints are satisfied.

Conceptual Question 4

Global constraint and examples

A global constraint is a high-level constraint involving many variables with a dedicated propagation algorithm that exploits its structure better than a decomposition into simple constraints.

Examples:

- `alldifferent`($X_1, \dots, X_n$)
- `cumulative` for scheduling
- `global_cardinality`

Conceptual Question 5

Symmetry breaking constraint and examples

Symmetry breaking constraints are additional constraints added to reduce symmetrical equivalent solutions in the search space.

Examples:

- Lexicographic ordering of rows in a matrix of decision variables.
- Fixing the first occurrence of a particular value.
- Requiring one solution to be lexicographically minimal among all symmetric variants.

Conceptual Question 6

Constraint propagation

Constraint propagation is the process of iteratively narrowing the domains of variables by enforcing local consistency with respect to the constraints, removing values that cannot be part of any global solution.

Conceptual Question 7

Domain reduction

Domain reduction refers to the elimination of infeasible values from variable domains as a result of propagation.

Conceptual Question 8

Backtracking search

Backtracking search is a depth-first search algorithm that incrementally assigns values to variables; when a partial assignment violates some constraint or leads to an empty domain, it backtracks by undoing assignments and trying alternatives.

Conceptual Question 9

Problems where CP is effective

CP is particularly effective in highly combinatorial problems with complex logical or scheduling constraints, such as timetabling, personnel rostering, project scheduling, configuration, and routing with rich side constraints.

Exercise: Scheduling A, B, C

Time slots: $\{1, 2, 3\}$, tasks A, B, C of duration 1.

Constraints:

- A before B.
- C not at slot 1.
- All different time slots.

1. Variables and domains.

$$A, B, C \in \{1, 2, 3\}.$$

Initial domains:

$$D(A) = \{1, 2, 3\}, \quad D(B) = \{1, 2, 3\}, \quad D(C) = \{2, 3\}.$$

2. Constraints.

- Precedence: $A < B$.
- No two tasks in the same slot: `alldifferent`(A, B, C).
- Forbidden time: $C \neq 1$.

3. Constraint propagation / domain reduction.

- Since there are three tasks and three slots with an `alldifferent` constraint, the assignments must be a permutation of $\{1, 2, 3\}$.
- Because $C \neq 1$, we have $C \in \{2, 3\}$.
- Since $A < B$, variable A cannot be 3, so $D(A) \subseteq \{1, 2\}$.

4. All feasible schedules.

- If $C = 2$, then $\{A, B\} = \{1, 3\}$ and $A < B$ gives $(A, B, C) = (1, 3, 2)$.
- If $C = 3$, then $\{A, B\} = \{1, 2\}$ and $A < B$ gives $(A, B, C) = (1, 2, 3)$.

Thus feasible schedules are

$$(A, B, C) = (1, 3, 2) \quad \text{and} \quad (1, 2, 3).$$

5. Backtracking search description. A typical backtracking search with propagation would:

1. Select a variable, for example A , and assign a value.
2. Propagate constraints to reduce domains of the remaining variables.
3. If a domain becomes empty, backtrack.
4. Continue until all variables are assigned consistently or all possibilities are exhausted.

6 Decomposition Methods

Conceptual Question 1

General idea

Decomposition methods exploit special structure of large optimization problems by splitting them into smaller, more manageable subproblems that are coordinated iteratively through cuts or columns.

Conceptual Question 2

Structural properties suitable for decomposition

Useful properties include:

- Block or nearly block diagonal structure in constraints or variables.
- Linking or complicating constraints that couple otherwise separable substructures.
- Decomposable subproblems sharing only a few linking constraints.

Conceptual Question 3

Benders decomposition: idea, formulation, algorithm

Consider a MILP partitioned into master variables x and subproblem variables y :

$$\begin{aligned} \min \quad & c^\top x + d^\top y \\ \text{s.t.} \quad & Ax + By \geq b, \\ & x \in X, y \in Y. \end{aligned}$$

The idea:

- Fix $x = \bar{x}$ and solve the subproblem in y .
- If the subproblem is feasible, derive an optimality cut.
- If the subproblem is infeasible, derive a feasibility cut.
- Add these cuts to the master problem.

A typical master formulation introduces a variable θ :

$$\begin{aligned} \min \quad & c^\top x + \theta \\ \text{s.t.} \quad & \text{Benders cuts on } (x, \theta), \\ & x \in X. \end{aligned}$$

Algorithm (high level):

1. Initialize the master problem.
2. Solve the master to get (x^k, θ^k) .
3. Solve the subproblem with $x = x^k$.
4. Add feasibility or optimality cuts.
5. Repeat until no violated cuts remain.

Conceptual Question 4

Dantzig–Wolfe decomposition: formulation and algorithm

Dantzig–Wolfe decomposition applies to LPs with block structure:

$$\begin{aligned} \min \quad & \sum_{k=1}^K c_k^\top x^{(k)} \\ \text{s.t.} \quad & \sum_{k=1}^K A_k x^{(k)} = b, \\ & x^{(k)} \in P_k, \quad k = 1, \dots, K. \end{aligned}$$

Each $x^{(k)}$ can be expressed as a convex combination of extreme points:

$$x^{(k)} = \sum_j \lambda_{kj} v_{kj}, \quad \sum_j \lambda_{kj} = 1, \quad \lambda_{kj} \geq 0.$$

Algorithm (column generation view):

1. Start with a small subset of columns.
2. Solve the restricted master LP.
3. Use dual prices to define pricing problems.
4. Add any improving column and repeat.
5. If no improving column exists, stop.

Conceptual Question 5

Differences between Benders and Dantzig–Wolfe

- Benders adds rows (cuts) to the master.

- Dantzig–Wolfe adds columns.
- Benders is often used when complicating variables are in the master.
- Dantzig–Wolfe is often used when substructures are naturally represented by patterns or routes.

Conceptual Question 6

Why decomposition is important for large-scale optimization

Decomposition methods allow large models to be solved more efficiently by exploiting structure, leading to smaller subproblems, specialized algorithms, and better memory and time performance.

7 Cutting Planes and Branch-and-Cut

Conceptual Question 1

What is a cutting plane and how is it generated?

A cutting plane is a linear inequality added to an LP relaxation that is valid for all integer feasible solutions but violated by the current fractional LP solution.

Cuts can be generated by:

- Algebraic methods such as Gomory cuts.
- Problem-specific valid inequalities.
- Separation algorithms that detect violated cuts.

Conceptual Question 2

What is a valid inequality?

A valid inequality for an integer program is a linear inequality satisfied by all integer feasible solutions, although it may exclude fractional LP solutions.

Conceptual Question 3

Branch-and-cut algorithm: steps and components

Branch-and-cut combines branch-and-bound with cutting planes:

- Solve the LP relaxation at a node.
- Generate violated cuts and add them.
- Reoptimize.
- If integral, update incumbent.
- If infeasible or not promising, prune.
- Otherwise branch.

Conceptual Question 4

How cutting planes improve LP relaxations

Cutting planes tighten the LP relaxation by removing fractional solutions that are not convex combinations of integer feasible solutions, thereby improving bounds.

Conceptual Question 5

Difference between Branch-and-Bound and Branch-and-Cut

- Branch-and-bound uses branching and LP relaxations.
- Branch-and-cut augments this with systematic cut generation.

Conceptual Question 6

Why cutting planes are important in large MILPs

Strong cutting planes can dramatically tighten relaxations, reduce the integrality gap, and lower the number of nodes explored.

8 Column Generation and Branch-and-Price

Conceptual Question 1

What is column generation?

Column generation is an algorithm for large LPs where only a subset of variables is considered initially; new columns with negative reduced cost are generated iteratively and added until no further improvement is possible.

Conceptual Question 2

What is the pricing problem?

The pricing problem is the subproblem that, given the current dual prices of the restricted master, searches for a new variable with negative reduced cost.

Conceptual Question 3

Difference between column generation and branch-and-price

- Column generation solves LP relaxations by adding variables iteratively.
- Branch-and-price integrates column generation into branch-and-bound for integer programming.

Conceptual Question 4

Problems where column generation is commonly used

Common applications:

- Vehicle routing problems with time windows.
- Crew scheduling.
- Cutting stock and bin packing.
- Set partitioning and set covering models with many patterns.

9 Algorithm Design: Flowcharts and Pseudocode

Dynamic Programming Algorithm for 0/1 Knapsack

Pseudocode

We use the $V(i, c)$ recurrence defined earlier.

Algorithm 1 Dynamic Programming for 0/1 Knapsack

Require: Profits $p_1, \dots, p_n$, weights $w_1, \dots, w_n$, capacity W

Ensure: Optimal value $V(n, W)$ and an optimal solution $(x_1, \dots, x_n)$

```
1: Create table  $V[0..n][0..W]$ 
2: for  $c = 0$  to  $W$  do
3:    $V[0][c] \leftarrow 0$ 
4: end for
5: for  $i = 1$  to  $n$  do
6:   for  $c = 0$  to  $W$  do
7:     if  $w_i > c$  then
8:        $V[i][c] \leftarrow V[i-1][c]$ 
9:     else
10:       $V[i][c] \leftarrow \max(V[i-1][c], V[i-1][c - w_i] + p_i)$ 
11:    end if
12:  end for
13: end for
14:  $c \leftarrow W$ 
15: for  $i = n$  downto  $1$  do
16:   if  $V[i][c] = V[i-1][c]$  then
17:      $x_i \leftarrow 0$ 
18:   else
19:      $x_i \leftarrow 1$ 
20:      $c \leftarrow c - w_i$ 
21:   end if
22: end for
23: return  $V[n][W], (x_1, \dots, x_n)$ 
```

Flowchart description

A flowchart for this algorithm should include:

- **Initialization:** create table V , set row $i = 0$ to zeros.
- **Outer loop:** iterate over items.
- **Inner loop:** iterate over capacities.
- **Decision:** check whether $w_i > c$.
- **Transition:** either copy previous value or compute the max of taking and skipping.
- **Backtracking:** reconstruct chosen items.
- **Termination:** output optimal value and selected items.

Generic Branch-and-Bound Algorithm for Maximization

Pseudocode

Algorithm 2 Generic Branch-and-Bound for Maximization

Require: Optimization problem $\max f(x)$ such that $x \in S$

Ensure: Optimal solution x^* and value z^*

```

1: Initialize incumbent value  $z^* \leftarrow -\infty$ , incumbent solution  $x^* \leftarrow \text{null}$ 
2: Initialize node list  $\mathcal{L}$  with the root node representing the whole feasible set  $S$ 
3: while  $\mathcal{L}$  is not empty do
4:   Select and remove a node  $N$  from  $\mathcal{L}$ 
5:   Compute an upper bound  $UB(N)$  by solving a relaxation of  $N$ 
6:   if  $UB(N) \leq z^*$  then
7:     prune node  $N$ 
8:   else
9:     if relaxation solution  $x^N$  is infeasible for  $N$  then
10:      prune node  $N$ 
11:     else if  $x^N$  is feasible for the original problem then
12:       Compute  $z = f(x^N)$ 
13:       if  $z > z^*$  then
14:          $z^* \leftarrow z$ 
15:          $x^* \leftarrow x^N$ 
16:       end if
17:     else
18:       Choose a branching variable and branching rule for node  $N$ 
19:       Generate child nodes  $N_1, N_2, \dots$ 
20:       Add children to node list  $\mathcal{L}$ 
21:     end if
22:   end if
23: end while
24: return  $x^*, z^*$ 

```

Flowchart description

A flowchart should show:

- **Initialization:** start, set incumbent, initialize node list with root.

- **Node selection:** check whether the list is empty; if yes, terminate.
- **Bounding:** solve a relaxation and compute the bound.
- **Pruning:** if the bound is not promising or the node is infeasible, prune.
- **Integrality check:** if the solution is integral, update the incumbent.
- **Branching:** if the solution is fractional, create child nodes.
- **Termination:** when the list is empty, output the incumbent solution.